

Week 1- Overview

Software engineering

The term **software engineering** is the made of two words, **software** and **engineering**.

Software is more than just a program code. A program is an executable code, which serves some computational purpose. Software is considered to be a collection of executable programming code, associated libraries and documentations. **Software**, when made for a specific requirement is called **software product**.

OR

Software is a collection of computer programs that when executed together with data provide desired outcomes.

Engineering on the other hand, is all about developing products, using well-defined, scientific principles and methods.

OR

Engineering is the application of **scientific** and **practical** knowledge to **invent, design, build, maintain,** and **improve frameworks, processes, etc.**



Software Engineering is an engineering branch related to the evolution of software product using well-defined scientific principles, techniques, and procedures. The result of software engineering is an effective and reliable software product.

OR

Software engineering is an engineering approach on a software development of systematic application.

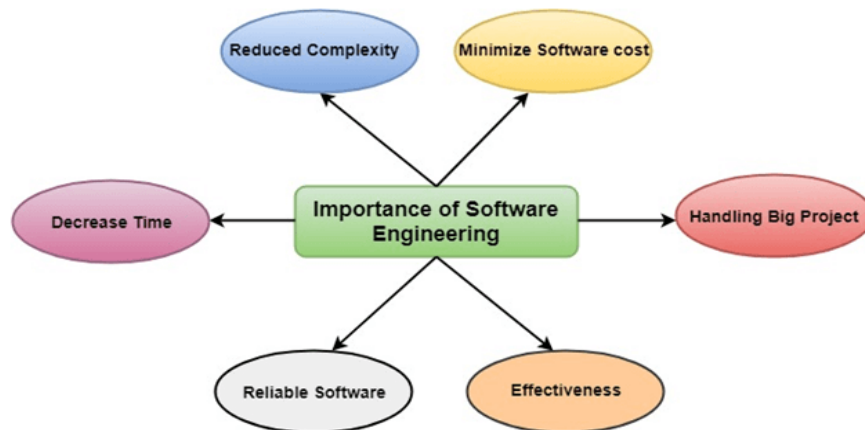
OR

Software engineering is an engineering, technological, and managerial discipline that provides a systematic approach to the development, operation, and maintenance of s/w.

Need of software engineering

The need of software engineering arises because of higher rate of change in user requirements and environment on which the software is working.

- 1. Large software** - It is easier to build a wall than to a house or building, likewise, as the size of software become large engineering has to step to give it a scientific process.
- 2. Scalability**- If the software process were not based on scientific and engineering concepts, it would be easier to re-create new software than to scale an existing one.
- 3. Cost**- As hardware industry has shown its skills and huge manufacturing has lower down the price of computer and electronic hardware. But the cost of software remains high if proper process is not adapted.
- 4. Dynamic Nature**- The always growing and adapting nature of software hugely depends upon the environment in which the user works. If the nature of software is always changing, new enhancements need to be done in the existing one. This is where software engineering plays a good role.
- 5. Quality Management**- Better process of software development provides better and quality software product.



The importance of Software engineering is as follows:

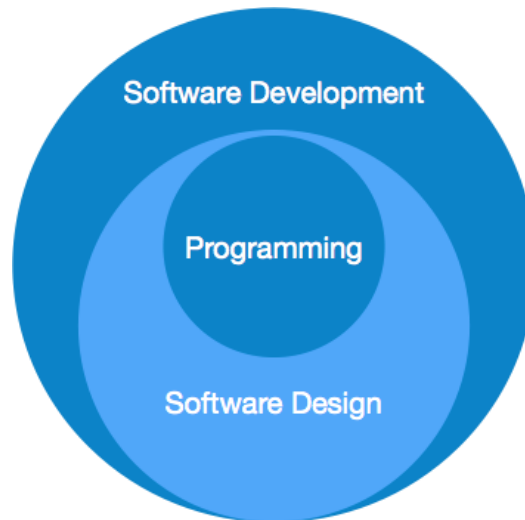
- 1. Reduces complexity:** Big software is always complicated and challenging to progress. Software engineering has a great solution to reduce the complication of any project. Software engineering

divides big problems into various small issues. And then start solving each small issue one by one. All these small problems are solved independently to each other.

2. **To minimize software cost:** Software needs a lot of hardwork and software engineers are highly paid experts. A lot of manpower is required to develop software with a large number of codes. But in software engineering, programmers project everything and decrease all those things that are not needed. In turn, the cost for software productions becomes less as compared to any software that does not use software engineering method.
3. **To decrease time:** Anything that is not made according to the project always wastes time. And if you are making great software, then you may need to run many codes to get the definitive running code. This is a very time-consuming procedure, and if it is not well handled, then this can take a lot of time. So if you are making your software according to the software engineering method, then it will decrease a lot of time.
4. **Handling big projects:** Big projects are not done in a couple of days, and they need lots of patience, planning, and management. And to invest six and seven months of any company, it requires heaps of planning, direction, testing, and maintenance. No one can say that he has given four months of a company to the task, and the project is still in its first stage. Because the company has provided many resources to the plan and it should be completed. So to handle a big project without any problem, the company has to go for a software engineering method.
5. **Reliable software:** Software should be secure, means if you have delivered the software, then it should work for at least its given time or subscription. And if any bugs come in the software, the company is responsible for solving all these bugs. Because in software engineering, testing and maintenance are given, so there is no worry of its reliability.
6. **Effectiveness:** Effectiveness comes if anything has made according to the standards. Software standards are the big target of companies to make it more effective. So Software becomes more effective in the act with the help of software engineering.

Software paradigms

Software paradigms refer to the methods and steps, which are taken while designing the software. There are many methods proposed and are in work today, but we need to see where in the software engineering these paradigms stand. These can be combined into various categories, though each of them is contained in one another:



Programming paradigm is a subset of Software design paradigm which is further a subset of Software development paradigm.

Software Development Paradigm

This Paradigm is known as software engineering paradigms where all the engineering concepts pertaining to the development of software are applied. It includes various researches and requirement gathering which helps the software product to build. It consists of –

- Requirement gathering
- Software design
- Programming

Software Design Paradigm

This paradigm is a part of Software Development and includes –

- Design
- Maintenance
- Programming

Programming Paradigm

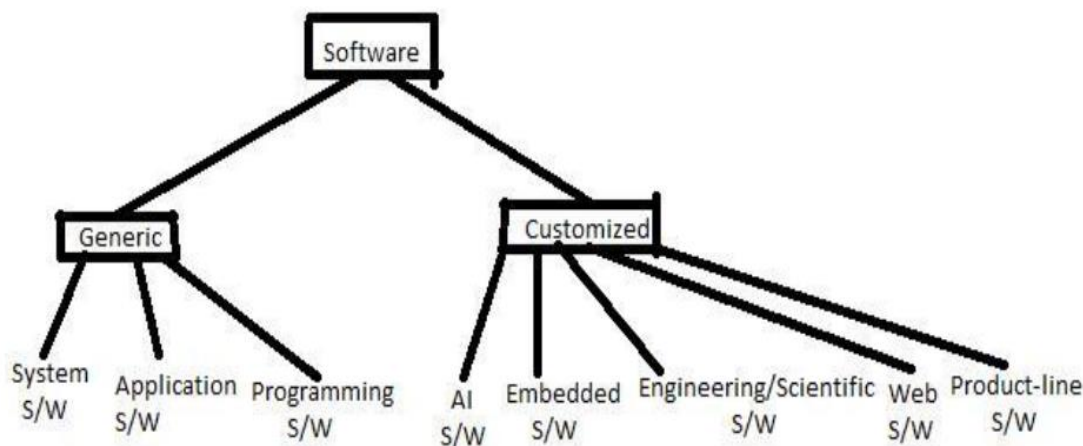
This paradigm is related closely to programming aspect of software development. This includes

- Coding
- Testing
- Integration

A **software paradigm** is a theoretical framework that serves as a guide for the development and structure of a software system. There are several software paradigms, including:

1. **Imperative paradigm:** This is the most common paradigm and is based on the idea that a program is a set of instructions that tell a computer what to do. It is often used in languages such as C and C++.
2. **Object-oriented paradigm:** This paradigm is based on the idea of objects, which are self-contained units that contain both data and behavior. It is often used in languages such as Java, C#, and Python.
3. **Functional paradigm:** This paradigm is based on the idea that a program is a set of mathematical functions that transform inputs into outputs. It is often used in languages such as Haskell, Lisp, and ML.
4. **Logic paradigm:** This paradigm is based on the idea that a program is a set of logical statements that can be used to infer new information. It is often used in languages such as Prolog and Mercury.
5. The **Software Development Life Cycle (SDLC)** is a process that software developers use to plan, design, develop, test, deploy, and maintain software systems. The most common SDLC models include:
6. **Waterfall model:** This model is based on the idea that software development is a linear process, with each phase building on the previous one.
7. **Agile model:** This model is based on the idea that software development is an iterative process, with small

Software product types: generic, customized;



1. System software:

It is a computer S/W which executes programs, transfers data between devices and controls, and operates the computer H/W.

Ex: Device Drivers, boot program, operating systems, servers, utilities, and so on. System S/W reduces the burden of Application programmers.

2. Application S/W:

It is designed to accomplish certain specific needs of the end user, by using the capabilities of the system S/W.

Ex: Enterprise s/w, sale transaction s/w, Educational s/w, video editing s/w, word processing, spreadsheet etc.

These may be available in different forms such as licensed, sold, freeware, shareware, open source, need to be installed, run online etc.

3. Programming S/W:

Is a class of system s/w which helps programmers in writing computer programs using different programming languages in a convenient manner.

Ex: Text Editors, Compilers, Interpreters, debuggers, linkers, and loaders etc, all in the form of a single application called IDE-Integrated Development Environment.

4. AI S/W:

It is made to think like human beings and therefore it is useful in solving complex problems automatically. It uses techniques or algorithms for working programs to represent and manipulate knowledge.

Ex: Game Playing, Speech recognition, understanding natural language, robotics etc are some AI applications.

5. Embedded S/W:

Type of s/w that is built in h/w systems. It is used to control, monitor or assist the operation of equipment, machinery, or plant.

Ex: 1. Applications or Daily Life Examples- Washing M/C's, cars, mobiles etc.

2. Controllers, real-time OS's, communication protocols are examples of Embedded S/W.

6. Engineering/Scientific S/W:

Engineering problems and quantitative analysis are carried out using automated tools.

Scientific S/W is typically used to solve mathematical functions and calculations. Computer- Aided

Design and Computer Aided Manufacturing S/W (CAD/CAM), civil engineering etc.

7. Web S/W:

These have been evolved from a simple website to search engines to web computing, which are spread over the N/W.

Web applications are based on client-server architecture, where the client requests information and the server stores and retrieves information from the web. Ex: Web 2.0, HTML, PHP, search engines

8. Product-Line S/W:

It is a set of S/W systems that a common, managed set of features to satisfy the specific needs of a particular market segment or mission. These are developed from a common set of core asserts in a prescribed way. Product line s/w improves time to market, productivity, quality and other business drivers. Applications: Multimedia, database s/w, word processing etc.

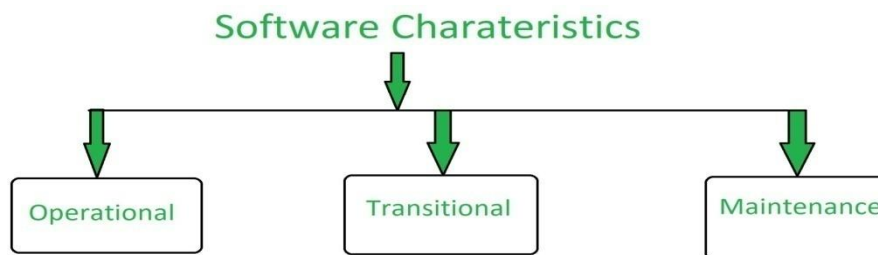
Sl.No.	Generic software product development	Custom software development
1.	The generic software development is done for developing a general purpose software.	Customer software development is done to develop a software product as per the needs of particular customer.
2.	In this development process, the software developers have to depict the end-users specifications.	In this development process, the the end-user requirements can be aggregated by communicating by them.
3.	From designing and marketing perspective, this type of development is very difficult.	This development does not require marketing, because it is developed for appropriate group of users.
4.	Large number of users may be using this kind of software.	This type of software is used by limited number of users.
5.	Quality of the product is not a preference for generic software.	Quality is the main criterion in customer software product. Best quality of the product is focused for customer or company.
6.	Development team controls the process of generic software development.	Customer determines the process of software development in this type of product.

Sl.No.	Generic software product development	Custom software development
7.	Generally the software developed is economical. There may be some hidden costs such as installation and implementation cost.	Software product is of high cost as the particular product for customer is developed.
8.	Example of generic software product development is Word-editing software.	Inventory control and management system are examples of customer software development.

Characteristics of good software

A software product can be judged by what it offers and how well it can be used. This software must satisfy on the following grounds:

- Operational
- Transitional
- Maintenance



1. Operational

In operational categories, the factors that decide the software performance in operations. It can be measured on:

- Budget
- Usability
- Efficiency
- Correctness
- Functionality
- Dependability
- Security
- Safety

2. Transitional

When the software is moved from one platform to another, the factors deciding the software quality:

- Portability
- Interoperability
- Reusability
- Adaptability

3. Maintenance

In this categories all factors are included that describes about how well a software has the capabilities to maintain itself in the ever changing environment:

- Modularity
- Maintainability
- Flexibility
- Scalability

Challenges in software projects

Challenge 1: Extremely high competition

If your software company has a great idea, chances are another company has already thought of it. The competition is extremely high both at the local and international levels, and it affects software businesses in terms of pricing, customer reach and retention, etc. PMs have to work closely with business owners and other stakeholders to identify the right market segment and ensure the ROI of their software.

Challenge 2: Old legacy systems

Software companies often spend significant resources on maintaining and upgrading the old legacy systems. Having invested a lot of financial and human resources, stakeholders become resistant and don't want to change the existing system, even when it no longer meets their needs. Instead of starting a battle and making stakeholders' more resistant, PMs should find ways to alleviate their fears and convince them of all benefits a new solution will bring to their business.

Challenge 3: High-level software expertise

When it comes to software selection and implementation, the best variant for business owners is finding project managers with the relevant software expertise. And the more complex the software system is, the more experience and the more specific skill set will be required for its implementation (e.g. think of large ERP systems).

Challenge 4: Third-party integration

Modern companies are no longer interested in standalone solutions and look for third-party integration. In general, it looks like implementing multiple systems in one project (e.g. a PM implements a financial management system with accounting and reporting modules which interface with CRM and contract management software). This puts PMs under pressure and makes them improve their expertise and learn more about other software that integrates with the solution they are implementing.

Challenge 5: Multiple-level users

Most companies look for systems that allow different types of users – from basic users to strictly IT users. Project managers who are responsible for the system implementation, must be familiar with all types of users and know what user rights and permissions should be assigned to each.

Challenge 6: Quality testing

Successful system implementation requires numerous testing iterations to ensure that the final outcomes align with the desired results. Project managers need to make sure all bugs are discovered and all issues are fixed before the system goes live. This is essential to avoid additional rework and ensure customer satisfaction.

Factors that influence software development;

1.People –

When we refer to people in software development, we actually mean hierarchy of roles in software development such as managers, team leaders, senior developers etc. They are ones who form development team and work in development of software. The order of priority of people among other dependency factors is **first** (highest). Software development is people-centric activity.

Software development requires good managers, people who understand psychology of people and provide good leadership.

A good manager does not ensure success of project, but can increase probability of success. A good manager gives priority to areas like proper selection, training, compensation, career development, work culture, development team's efficiency etc.

2.Product

A product is solution of problem that is handed to user who is struggling with problem. The priority of product among other dependency factors is **second** that is after **people**. The product or solution to problem is constructed based on requirements, which are objectives and scope work of problem. This

helps managers to select best approach within constraints imposed by delivery deadlines, budgetary restrictions, personnel availability, technical interfaces etc. Without well defined requirements, it may be impossible to define reasonable estimates of cost, development time and schedule for project.

3.Process

The process is way in which we produce software. It provides framework from which comprehensive plan for software development can be established. If process is weak, end product will undoubtedly suffer. The priority of process among other dependency factors is **third**, after **people** and **product**, however, it plays very critical role for success of project. There are many life cycle models and process improvements models. A suitable process model is determined on basis of requirements specification, development team, user, type of project and associated risks of project.

4.Project

A proper planning is required to monitor status of development and to control complexity. For project to be successful, it should be within budget, follow policies and requirements defined, and completed within timeline stated. Most of projects overrun cost by 100% of budget and are not delivered on time. The priority of project among other dependency factors is **last** (lowest). In order to manage successful project, we must understand and prevent associated risks. Concrete requirements should be defined and frozen, wherever possible. Unnecessary changes should not be made to avoid software surprises. Software surprises are always risky and should be minimized. A planning mechanism should be constructed to caution from occurrence of any surprises.

Project Success:

Define Success:

- ✓ Project Success can be defined as the achievement of something desired, planned or attempted.
- ✓ It is also said that success is an event that accomplishes its intended purpose.
- ✓ Anything short of that is failure.

Some Causes of Project Success:

1. Consider end users and implement user testing.
2. Have clear objectives and specifications.
3. Hire experienced developers.

4. Have a plan
5. Practice effective project management.
6. Have clear communication.
7. Focus on your design.
8. Implement risk management.
9. Follow software development best practices.
10. Do a final review and post-release review.

Software project success

A Software Project is successful if it delivers the product with pre-agreed level of quality within the given time and cost.

You can measure the success of a project by reviewing project scope, evaluating project specifications, analyzing the project budget, and reviewing client and internal satisfaction.

These steps are described below.

Step 1: Review the Scope of the Project

Start by reviewing the requirements and deliverables that the team agreed upon at the beginning of the project. For each deliverable, note the required resources, tasks, and the time it took to complete.

Step 2: Evaluate the Project Specifications

Did the project meet the expectations and goals? You can determine this by assessing whether or not all the deliverables are completed and agreements are met within the expected timeframe.

Step 3: Analyse the Project Budget

The budget can be an essential indicator of project success. Look at the amount of time spent on the project, and whether or not it exceeded the budget.

Step 4: Review Client Satisfaction

Client satisfaction with the project result is vital. If they are not fully satisfied, they must be allowed to provide feedback, which you can take as insight for future projects to help generate repeat business.

Step 5: Review Internal Growth and Team Satisfaction

It's important that the project team feels good about their role and the project's result. Was the project challenging and interesting? Is the team satisfied with the quality of the deliverables? These factors contribute to the project team's sense of fulfilment and job satisfaction

Software process:

A software process is a structured set of activities required to develop a software system.

It is a set of ordered activities carried out to produce a software product.

Each activity has well-defined objective, task and outcome.

An activity is a specified task performed to achieve the process objectives. The outcomes of all activities are compiled and integrated together to design the s/w.

The development of software is done with the help of some s/w process methodologies.

Each activity of a software process involves tools and technologies (CASE tools, compiler etc), procedures (algorithms, installation procedure etc.) and artifacts (the intermediate of final outcomes).

OR

A software process (also known as software methodology) is a set of related activities that leads to the production of the software. These activities may involve the development of the software from the scratch, or, modifying an existing system.

Any software process must include the following four activities:

1. **Software specification** (or requirements engineering): Define the main functionalities of the software and the constraints around them.
2. **Software design and implementation**: The software is to be designed and programmed.
3. **Software verification and validation**: The software must conform to its specification and meet the customer's needs.
4. **Software evolution** (software maintenance): The software is being modified to meet customer and market requirements changes.

Software processes in software engineering refer to the methods and techniques used to develop and maintain software. Some examples of software processes include:

1. **Waterfall**: a linear, sequential approach to software development, with distinct phases such as requirements gathering, design, implementation, testing, and maintenance.
2. **Agile**: a flexible, iterative approach to software development, with an emphasis on rapid prototyping and continuous delivery.
3. **Scrum**: a popular Agile methodology that emphasizes teamwork, iterative development, and a flexible, adaptive approach to planning and management.
4. **DevOps**: a set of practices that aims to improve collaboration and communication between development and operations teams, with an emphasis on automating the software delivery process.

Components of Software

There are three components of the software: These are: Program, Documentation, and Operating Procedures.

1. Program –

A computer program is a list of instructions that tell a computer what to do.

2. Documentation –

Source information about the product contained in design documents, detailed code comments, etc.

3. Operating Procedures –

Set of step-by-step instructions compiled by an organization to help workers carry out complex routine operations.

Components of Process

1. Events: Events are the conditions which must exist for the process to be performed. It is something that happens as opposed to something that is done on purpose. It can think of as the effect which occurs after sufficient cause is provided. Each process starts and ends with an event.

2. Tasks: A task is the smallest unit into which the activity can be broken down. Breaking it down is not feasible for the purpose at hand. The business process describes the different activities as well as the interrelationship between them. It is important to note that inter-relationships are more important than the tasks. In any structure, the whole is greater than the sum of its parts. While conducting BPM exercises, one must therefore have a synergistic view.

The number one problem with BPM today is that most of the practitioners are unable to understand the system viewpoint. The employees of the same business have conflicting objectives. Therefore a human resource professional may end up optimizing their process, but may have an adverse effect on the functioning of a marketing department. Thus problems are merely being shifted than actually being resolved. A good understanding of how the process connects to other activities and processes will help solve this problem and achieve sustainable progress.

3. Decisions: There might be certain decisions which may have to be taken as the part of a process. Leaving the decisions up to the people involved has undesirable consequences. It is likely that in the absence of clear guidelines, the decisions taken by different people will be different. This will create inconsistent experiences for the customers and bring down quality.

As an example, consider a leave granting process in any big organization. There are explicit rules which define the number of leaves that a person can take as well as the procedure to get them approved. Thus although it may look like the manager is taking the decisions with regards to granting leave, all they are doing is following a pre-defined procedure. Thus no matter who the manager is, the decisions will always remain consistent because they are taken on the basis of rules rather than on the basis of who is involved. Such rules are usually laid down as if, then and else conditions in the process.

4. Inputs: Until gives inputs, a process cannot function. The correct inputs are like the correct food for the process. Just like eating unhealthy food makes us unhealthy, giving wrong inputs makes the process unhealthy and inefficient. Here are some common inputs required by a process.

a. People: Processes require people with the correct aptitude and attitude. This is why breaking down of tasks is so important. In a process driven organization, you can arrange for an unskilled person to do the mundane jobs while a skilled person can be deployed to do the important jobs. Matching skills with task requirements bring down costs and increases efficiency.

b. Raw Material: Raw materials need to be made available in a timely manner and at least costs. There are companies which have built procurement processes as their core competencies.

c. Information: The correct information needs to be made available to all the entities in the process. The worker must have the skill and must be well versed with the procedure. The manager must get continuous feedback to ensure that the production is on target and as planned.

5. Outputs: The outputs from the process must be continuously monitored. This will help in measuring the effectiveness and efficiency of the process and suggesting changes as and when required.

What are software process activities?

A software process (also known as software methodology) is **a set of related activities that leads to the production of the software**. These activities may involve the development of the software from the scratch, or, modifying an existing system.

Process Activities



1. Software **specification** - defining what the system should do;
2. Software **design and implementation** - defining the organization of the system and implementing the system;
3. Software **validation** - checking that it does what the customer wants;
4. Software **evolution** - changing the system in response to changing customer needs.

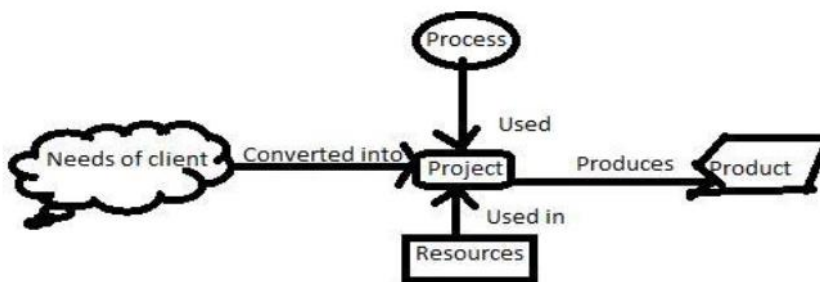
Difference Product, project and Process

Software Product: It is the outcome of the s/w project produced through processes. A project can have more than one product called work products. A work product is the intermediate outcome of the processes. But the final work product is referred to as a product or S/W.

Software Project: It is an entity. With defined start and, in which a S/W process is being used. Type of the process and its execution depend on the nature of project.

Software Process: It is a set of ordered activities carried out to produce a software product.

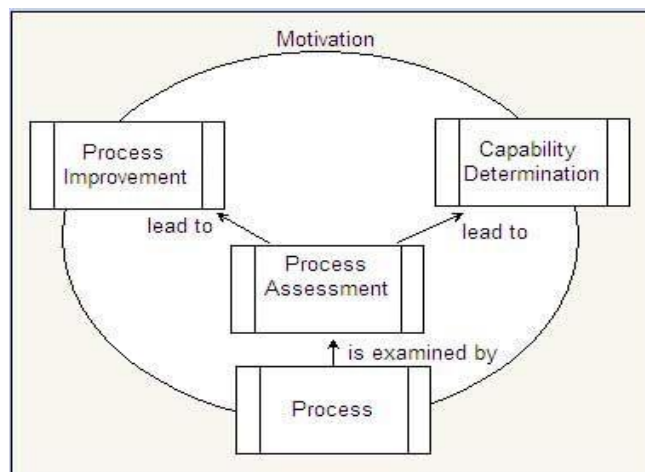
Diagrammatical Representation of the Relationship b/w Process, Project and Product:



The following table highlights the important differences between a Product and a Process –

Product	Process
Product is the final production phase in the project.	Process refers to a set of sequence of steps that should be followed with the goal of creating a project.
Product focuses on the final result.	Process focuses on completing every step involved in the project that is being developed.
Product follows the firm guidelines.	Process follows the guidelines consistently.
Product tends to be a short-term aspect.	Process tends to be a long-term aspect.
The goal of a product is to complete the work successfully.	The goal of a process is to improve the quality of the project.

Process Assessment and improvement:



The process assessment leads to process capability determination and process improvement. Process capability determination is an organized assessment, which analyzes the software processes in an organization. Assessment includes five phases initiating, diagnosing, establishing, acting and learning. SPICE (Software Process Improvement and Capability Determination) is a standard used for both process improvement and process capability determination. SPICE provides a framework for assessing the software process and is used by the organizations involved in planning, monitoring, developing,

managing, and improving acquisitions. It is carried out in accordance with the **International Organization for Standardization (ISO)** and **International Electro-technical Committee (IEC)**, which are used together and known as **ISO/IEC 15504**.

Software processes are assessed to ensure their ability to control the cost, time and quality of software. Assessment is done to improve the software process followed by an organization.

Software Process Improvement (SPI) Cycle includes:

- ✓ Process measurement
- ✓ Process analysis
- ✓ Process change

Software engineering ethics:

Like other engineering disciplines, software engineering is carried out within a social and legal framework that limits the freedom of people working in that area. As a software engineer, you must accept that your job involves wider responsibilities than simply the application of technical skills. You must also behave in an ethical and morally responsible way if you are to be respected as a professional engineer. It goes without saying that you should uphold normal standards of honesty and integrity. You should not use your skills and abilities to behave in a dishonest way or in a way that will bring disrepute to the software engineering profession. However, there are areas where standards of acceptable behavior are not bound by laws but by the more tenuous notion of professional responsibility. Some of these are:

1. **Confidentiality** You should normally respect the confidentiality of your employers or clients regardless of whether or not a formal confidentiality agreement has been signed.
2. **Competence** You should not misrepresent your level of competence. You should not knowingly accept work that is outside your competence.
3. **Intellectual property rights** You should be aware of local laws governing the use of intellectual property such as patents and copyright. You should be careful to ensure that the intellectual property of employers and clients is protected.
4. **Computer misuse** You should not use your technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine) to extremely serious (dissemination of viruses or other malware).

Professional societies and institutions have an important role to play in setting ethical standards. Organizations such as the ACM, the IEEE (Institute of Electrical and Electronic Engineers), and the British Computer Society publish a code of professional conduct or code of ethics. Members of these organizations undertake to follow that code when they sign up for membership. These codes of conduct are generally concerned with fundamental ethical behaviour.

Software engineering ethics can be approached from three directions. First, it can describe the activity of software engineers making practical choices that affect other people in significant ways. Second, it can be used to describe a collection of principles, guidelines, or ethical imperatives that guide or legislative action, and third, it can be used to name a discipline that studies the relationship between the other two senses of ethics.

Software engineering ethics is clearly both an activity and a body of principles. The discipline of software engineering ethics that studies this activity and formalizes these principles, however, is in its infancy

Software engineers shall adhere to the following Eight Principles:

PUBLIC - Software engineers shall act consistently with the public interest.

CLIENT AND EMPLOYER - Software engineers shall act in a manner that is in the best interests of their client or employer and that is consistent with the public interest.

PRODUCT - Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.

JUDGMENT - Software engineers shall maintain integrity and independence in their professional judgment.

MANAGEMENT - Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.

PROFESSION - Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.

COLLEAGUES - Software engineers shall be fair to and supportive of their colleagues.

SELF - Software engineers shall participate in lifelong learning regarding the practice of their profession and promote an ethical approach to the practice of the profession.

Define Failure:

- ✓ A failure can be defined for a task or project or assigned work.
- ✓ Failure for a project is, “that fails to perform a duty or an expected action, non-occurrence or non-performance”.

Startling Facts:

Only 2.5% of companies in South Africa successfully completed 100% of their projects in the last 2 years. The average cost overrun of a Project is 27%

- ✓ 57% of projects fail due to “breakdown in communications.” (No 2 cause of Project Failure at the moment)
- ✓ 39% of projects fail due to lack of planning, resources, and activities.
- ✓ 60% of failed projects have a duration of less than one year.
- ✓ 31% of Development Programme’s are cancelled
- ✓ 53% of Projects have a cost overrun over 180 % over their Original Budget.
- ✓ By 2020 , 40% of existing infrastructure programmes will be stopped due to poor execution
- ✓ By 2025, 45% of firms appointed as Consultant Project Managers will have failed their Clients.

Some Causes of Project Failures:

- ✓ **Lack of senior management support**- happens when “Junior” or “Candidate PMs” are thrown into the Project and left unattended.
- ✓ **Poor communication**- causes delay or even failure since team members do not have the information they needed, issues or changes do not get escalated, project reporting is sluggish
- ✓ **Inadequate resources**- Task take longer than expected to complete, deadlines and milestones get missed, and project completion date comes into jeopardy, one end of working more than necessary (double shift) to get the work done No one is in control not even the project manager, who is assigned to the project but not given the free hand to manage the project.
- ✓ **Poor definition of Scope**- Project changes from its original objective and goals. This can occur due to additional requirement from the client.
- ✓ **Project lacks structure** - caused by things such as critical tasks being underrated
- ✓ **Inaccurate estimates** - A top- down plan causes constraints on the prediction of the cost of the project
- ✓ **Poor risk management** - The project planning stages (Stage 1 – 3) is not properly planned
- ✓ **Unrealistic Milestones** - it is not possible for a project to succeed if unrealistic timeframes and milestones are made available for that project
- ✓ **Incompetent project management skill** - procedures and policies are not adhered to from inception